

Sign Language Recognition Using CNN

Michael Tesis
AI 700-001
Long Island University
Brooklyn, New York, USA
Michael.Tesis@my.liu.edu

Robert Mullis
AI 700-001
Long Island University
Brooklyn, New York, USA
Robert.Mullis@my.liu.edu

Abstract — From an image containing a letter in American Sign Language, we use a CNN model to recognize the letter in the image and output it as text. Our CNN model preprocesses the image in the convolutional layers with self-learning filters, padding, and pooling. Then the training is completed in the deep neural network. The model performs multi-class classification to predict the best letter candidate for a given image. We demonstrate the capabilities of the chosen Deep Learning algorithm to perform SLR with a high accuracy.

Keywords — SLR, CNN, Deep Learning, filters, pooling

I. INTRODUCTION

The idea of Artificial Intelligence is to create a machine capable of rational thought and decisions like the human brain. The human brain receives most of its information using the percept of vision. For that reason it is of utmost importance to solve the problem of recognizing information about the outside world by an AI model similarly to human's sense of vision. Machine Learning is a subfield of AI that takes a step toward processing data obtained from the outside world, but lacks the ability to extract features from that data needed for accurate predictions. Deep Learning, which is a subfield of Machine Learning that has the ability to extract features, solves this problem and is the closest solution so far to being able to see the real world like humans do. One particular Deep Learning model that is the best suited for image recognition is the Convolutional Neural Network (CNN). While a Deep Learning model is a neural network containing one or more hidden layers made of neurons, CNN has preprocessing layers called convolutional layers that are able to extract features from images to prepare them for recognition by a deep neural network. CNN models have been proven to achieve high accuracies in image recognition. Our CNN model is tailored specifically toward recognizing American Sign Language letter images.

Sign Language is a precious tool developed for people who have hearing or speaking problems, which allows them to communicate. However, not everyone understands or knows Sign Language leading to the requirement of translation. Traditionally, this task is done by specially trained interpreters that may not always be available or present when needed. Having a machine that can perform translation of Sign Language would greatly improve life quality and communication for people who need it. Sign Language Recognition (SLR) is the machine recognition of hand signs in images or videos that outputs the corresponding meaning as text. SLR is not as mature of a field as other image recognition fields, such as Optical Character Recognition (OCR). However, the demand for it is high and the importance of its development is correspondingly strong. We hope that our work contributes to the development of SLR.

Most problems encountered in SLR correspond to general image recognition problems. One issue for image recognition is so called "noise" which means that the image contains unwanted details that obscure the contents of the image. When noise exists in an image, it is difficult for the SLR model to distinguish between hand signs and their surrounding background. Another problem is the similarity of different hand signs between each other that leads to the SLR model misrecognizing one letter for another. Even more problems include distortion and skew in images, low contrast between the meaningful part of the image and its background, distinguishing between hand signs and other objects in images, and recognizing hand signs under different lighting conditions and different viewing angles.

All of these problems, though not simple, are solvable using a CNN model. Such a solution takes advantage of the models ability to extract and learn features, which is the process of finding meaningful shapes and edges common to each hand sign letter class. This allows for accurate separation between the contents of the image and the background. Furthermore, the convolutional layer might contain filters that reduce the noise and improve image quality when necessary. Having a large enough dataset of images for training deals with the problem of different lighting conditions and viewing angles. Overall, CNN offers the techniques and algorithms needed to achieve high accuracy in Sign Language Recognition.

II. TASK DEFINITION

The goal of our project is to create an SLR program implementing a CNN model that will read an image containing a hand sign of a letter and output the letter in text format. The outputted letter should match the corresponding sign in the image with a high accuracy.

In our work, we created and tested an SLR program using a Deep Learning approach utilizing a CNN model. Our model has several convolutional layers responsible for preprocessing the image by extracting features and a neural network with a varying number of hidden layers. At first the model has to be trained, which is done through multiple iterations of forward passes and backpropagations. On each iteration the true labels are compared to the computational results obtained during the forward pass and the difference is calculated as a loss function. The computational results obtained during a forward pass depend entirely on the parameters called weights and biases. The gradients of the log loss function are calculated in order to minimize the loss and the weights in both the convolutional and neural network layers are updated. Training continues until a minimal value of the loss function or the preset number of iterations is obtained. The end goal of model training is to obtain optimal values for the weights that consistently produce high accuracy recognition. After the training, the learned model is evaluated using images in the test dataset.

In general the effectiveness of a model is determined by two critical parts: the model architecture and the data used to train and test the model. The dataset plays an especially important role in Deep Learning models, which highly depend on it. Deep Learning models are often the best choice to process big datasets with a large number of classes and amount of data itself. They also outperform other AI and Machine Learning models when processing datasets smaller in size but with more complex data. Images are good examples of big complex data since each image is defined not by one or several numbers but by a matrix of pixels, which may be very large depending on the image size and resolution. Video streams contain even more data as when processed they are split into a large amount of frames, which are static images. The scope of our model is limited to processing images, not videos. Correspondingly, we chose a dataset containing images of hand signs for training and testing purposes. This limitation is quite significant for SLR as two letters out of twenty-six in American Sign Language are represented by the movement of a hand, which would require video to recognize them. If our model demonstrates good results recognizing images we might consider extending it to process videos in perspective.

III. INFRASTRUCTURE

In order to implement the model, we chose to use Python as the language for our program, VS Code as our development environment, and GitHub for version control and information exchange between team members. To implement different tasks we used the following Python libraries: TensorFlow, Pandas, NumPy, and Matplotlib. TensorFlow contains ready to use functions to create a model, train it, specify model parameters, and test it. We used Pandas to read images from the dataset that are stored in csv spreadsheets. NumPy contains implementations of many widely used mathematical functions such as argmax. Finally, Matplotlib is necessary to create graphs and plots. We use the Sign Language MNIST dataset [1]. It contains 27,455 training images and 7172 testing images that are in 28x28 pixel grayscale format. Each image is a picture of a hand showing a letter in American Sign Language with variations corresponding to different lighting conditions, different backgrounds, varying viewing angles, etc. The output of each program run is a letter that represents the prediction made by the model based on the hand sign image inputted for recognition. To accelerate the training we use a PC with a high performing GPU that TensorFlow can take advantage of.

IV. LITERATURE REVIEW

As a good starting point, paper [2] gives a comprehensive review of the history of SLR, its current status, and research papers that contributed to its development. Thus, it states that exploration into recognizing American Sign Language (ASL) first began with a research paper published in the mid-90s [3]. Since then there have been a large number of research papers world-wide that studied SLR implementations for their corresponding languages. Deep Learning technologies opened a new era in the development of the field. Due to their capabilities of processing large complex data and extracting features, they offer incomparable opportunities for progress in SLR. In particular, CNN-based approaches have achieved

state-of-the-art performance on several benchmark datasets [4]. During our research, we found three very recent papers related to our project.

The first paper [5] presents the results of the past five years of research into advancements, challenges, and opportunities in deep learning-based SLR. The authors explored such aspects of SLR as sign data acquisition technologies, sign language datasets, evaluation methods, and different types of neural networks. They come to the conclusion that Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) are the most effective models for static sign language recognition, which corresponds to the scope of our project. The paper provides a detailed description of a CNN's structure and components. The paper lists the main components of CNNs as convolutional layers, pooling layers, and fully connected layers. In our model we also use padding, a technique not mentioned in the paper. An image is fed into the input layer. Then the convolutional layer performs operations to extract local features. This is done in the following manner. A filter, also called a convolutional kernel, which is a small square matrix of numbers is slid over the input image and the dot product between the filter and the area it covers at the moment is computed. The result of applying a filter is a feature map, so the whole process corresponds to extracting features. The output of the convolutional layer is passed through a non-linear transformation using an activation function, most often ReLU (Rectified Linear Unit), to introduce non-linearity. The pooling layer reduces the size of the feature maps, retaining only important features. Then the output of the pooling layer becomes the input into the fully connected neural network. The last layer in the network is the output layer that produces the classification result. There are various datasets and models provided in the paper related to continuous rather than static inputs, which can be used for video stream recognition. This will be useful in our later work if we extend our model to processing videos. Together with the advances in SLR, the authors name challenges remaining in the field. The challenges mentioned in the paper include expanding sign language datasets, achieving user independence in recognition systems, improving semantic and syntactic understanding, and others. The paper concludes by emphasizing the necessity of further SLR development in order to address the previously discovered and newly emerging challenges.

The second paper [6] presents an evaluation of the performance of different deep neural network architectures in the recognition of sign language. From our point of interest, it references the research [7] that employed CNNs for feature extraction and classification. Guided by the utilization of pre-trained CNN models like "AlexNet" and "VGG-16", the research described the remarkable achievement of an accuracy rate of up to 98% in sign language classification. Other research papers referenced in [6] also prefer CNN models for static image classification. Therefore, instead of comparing the CNN model to other AI models, the paper focuses on comparing different architectures of CNN models. They mainly differ by the way hidden layers connect to each other. For example, Densenet [8], which is a CNN where every hidden layer is connected to all previous and future layers, was evaluated as the best architecture as it produced the highest recognition accuracy of 99%. Overall, the paper is a good source of methodologies to evaluate model effectiveness.

The third paper [9] provides a comprehensive overview of models, methodologies, and challenges in SLR based on the 32 research papers that the authors studied and reviewed. The paper mostly focuses on continuous sign language recognition (CSLR) that we don't use at the current phase of our project but may consider using in the future. In particular, the paper found the following limitation and research gaps that serve as pointers for future developments in the field. The main limitations come from the lack of existing datasets with high quality data and the difficulties of creating new ones that require a lot of resources and expenses. The authors point out to the necessity of developing models capable of working efficiently with small training datasets. Another suggested solution is to develop techniques for data generation, such as Generative Adversarial Networks (GANs). Other limitations relate to conditional changes during the gathering of data and testing, such as different signers producing variations in hand signs. Again, we don't currently address these challenges in our model since it is not continuous SLR, but it is good to know about them considering our future plans.

V. APPROACH

A. Our CNN Model

The input layer to our model contains a number of nodes that correspond to the number of pixels in the input image. This is defined by the image format in the dataset that we will describe later. From the input layer the data feeds into the convolutional layer where feature extraction from the image occurs using padding, filtering, and pooling. The padding adds pixels with 0 values around the perimeter of the input image in order for the image to retain its size after filtering. Filtering is done in order to detect patterns or features, such as edges, corners, and curves. This highlights the important shapes in the image and separates them from background and other meaningless objects that significantly improve recognition. In our model we used two sets of 3x3 dynamic filters. The first set of 32 filters detects simple features (edges, lines). The second set of 64 filters detects more complex patterns (shapes, curves, letters). They are called dynamic because their matrices are not set, but change during model training the same way as the weights in the fully connected layer. Our model allows the addition of more filters as we describe in the Further Development section. Each filter is slid across an image moving with a stride of 1. The stride of the filter indicates how many pixels to the right and down the filter moves on each step of computation. The result of the computation is a feature map and the computation itself uses dot multiplication shown by formula (1) in the Equations section. Then the ReLU activation function described by formula (2) is applied to the feature map in order to add non-linearity to the features learned. Since the resulting feature map is the same size as the input image, in most cases it is unnecessarily large and contains both important and not important features. To retain only important features in the map, pooling is used. Pooling is done using kernels similar to filters, but smaller in size, which slide across the feature map and compress the map. We use the MaxPooling function that builds a new compressed feature map by selecting a pixel with the largest value inside each area that it slides over.

From the convolutional layer the data is passed to the fully connected layer. In our model we use one hidden layer and a multi-class output layer. Our hidden layer contains 128 neurons that are all connected to every input and every output. Each neuron takes the input from the feature map as a vector, computes the result using ReLU activation function described by formula (2), and sends it to every neuron in the output layer. The output layer contains 26 neurons which correspond to 26 possible output classes. Each class represents a letter in the English alphabet. The resulting output is a vector of size 26 with each place in the vector containing the probability of the corresponding class. This vector is obtained using the Softmax activation function described by formula (3).

At first the model goes through training over a number of iterations (epochs), which we set to 15. In each iteration the input image goes through all of the layers described above in a forward pass and ends up with a predicted class. This result is compared to the true class specified in the training image label as a one-hot encoded vector. Next, the predicted result and the label are used to compute the log loss function, also known as categorical cross-entropy loss, described by formula (4). The goal of the training is to find weight and bias parameters, such that the loss function is minimized. This is achieved by calculating the gradients of the loss function with respect to the weights and biases. The gradients are found by calculating partial derivatives over weights, biases, and results of all layers using the chain rule starting from the output layer and going back to the input layer. This process is known as backpropagation [10]. After a forward pass and backpropagation the weights and biases are updated in all layers including the convolutional layers and the new values are used in the next iteration. How the new weights and biases values are obtained is described by formulas (5) and (6). In our model the training is completed after 15 iterations. For efficiency, our model processes input images as batches of 64 at the same time in parallel.

The last part of our model is the evaluation phase that predicts the classes of images in the testing dataset. Each prediction is compared to the label of the corresponding image in order to calculate the overall accuracy of the model as described by formula (7). The evaluation phase only goes through the forward pass and does not use backpropagation.

B. Equations

On each stride during filtering the feature map is calculated by the formula:

$$y = \sum_{i=1}^m \sum_{j=1}^n X_{i,j} \cdot W_{i,j} + b \quad (1)$$

In this formula $\mathbf{W}$ is a matrix for the filter kernel that is multiplied using the dot product with the $\mathbf{X}$ matrix representing the area of the image covered by the filter at the current stride. b is the number representing the bias for each filter needed to avoid zero values. This formula describes the per element multiplication that is the number in the filter multiplied by the number in the image in the corresponding matrix position and then all of the products are summed together to produce the new matrix that is the feature map.

The computation in each neuron in hidden and convolutional layers is performed using the ReLU activation function below that takes $\mathbf{W} \cdot \mathbf{X} + \vec{b}$ as the input parameter:

$$\text{ReLU}(x) = \begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases} \quad (2)$$

In this formula x is replaced with $\mathbf{W} \cdot \mathbf{X} + \vec{b}$ where $\mathbf{W}$ is the weights matrix, $\mathbf{X}$ is the matrix representing the feature map, and $\vec{b}$ is the biases vector. ReLU outputs $\mathbf{W} \cdot \mathbf{X} + \vec{b}$ if it is greater than 0 and outputs 0 otherwise.

The computation in each neuron in the output layer is performed using the Softmax activation function below that takes $\mathbf{W} \cdot \mathbf{X} + \vec{b}$ as the input parameter:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \quad (3)$$

In this formula $z_i = W_i x + b_i$ in the output node i where x is the output from the hidden layer, W_i is the weight of the node, and b_i is the bias of the node. The denominator in the formula is the sum of e to power of every $z_j = W_j x + b_j$. $\hat{y}_i$ is the probability of the class at node i . C is the number of classes which is 26 in our case. 26 output nodes produce a result in the form of a vector with the probabilities of each class at the index of each class.

The log loss function is described the formula:

$$L = - \sum_{i=1}^C y_i \cdot \log(\hat{y}_i) \quad (4)$$

In this formula y_i is the value at index i of the one-hot encoded vector. $\hat{y}_i$ is the probability of the class at index i in the output vector. C is the number of classes. L is the loss.

The new values for the weights and biases in all layers are obtained using the formulas:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w} \quad (5)$$

$$b_{new} = b_{old} - \eta \frac{\partial L}{\partial b} \quad (6)$$

In these formulas w_{new} and b_{new} are the new values of weights and biases, w_{old} and b_{old} are the old values, η is the learning rate, and L is the loss.

The accuracy of the model is calculated using the formula:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N 1(\hat{y}_i = y_i) \quad (7)$$

In this formula N is the total number of images in the train or test set. y_i is the true class label for image i . $\hat{y}_i$ is the predicted class label by the model. $1(\hat{y}_i = y_i)$ is an indicator function that is 1 if the prediction is correct and 0 if it is wrong. The Accuracy represents the fraction of correctly predicted images by the model out of all evaluated images.

C. Training and Testing Datasets

As mentioned in the Infrastructure section, we used the Sign Language MNIST dataset from Kaggle in our model. This dataset contains two spreadsheets with 27,455 training images and 7172 testing images. Each row in the spreadsheets contains the label of the image in the first column and the image pixel values in the next 784 columns. Each pixel value is in the range from 0 to 255. This corresponds to an image of 28 x 28 pixels in grayscale format. This dataset is a result of collecting hand sign images of each letter in the American Sign Language without any images of the letters j and z as they require hand motion. The images of each hand sign are varied by different lighting conditions, different backgrounds, different viewing angles of the hand sign, etc. Even though it is a relatively small dataset for a CNN model, our current results demonstrate that the model is well trained using it. New images will need to be converted to the dataset format.

VI. RESULTS

The decrease of the loss function during training and testing phases is shown on the graph below:

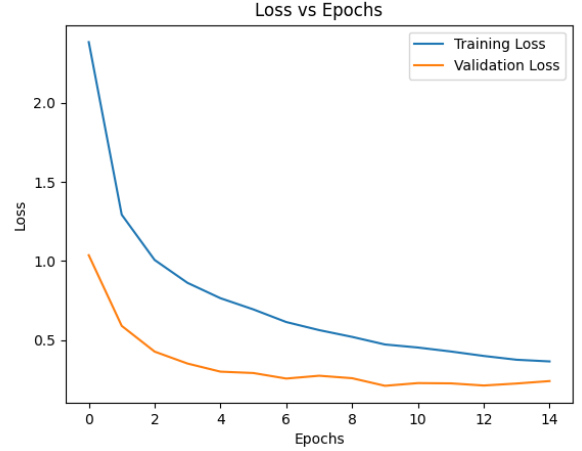


Fig 1. Dependency of loss on the number of iterations.

The increase of the accuracy of the model during training and testing phases is shown on the graph below:

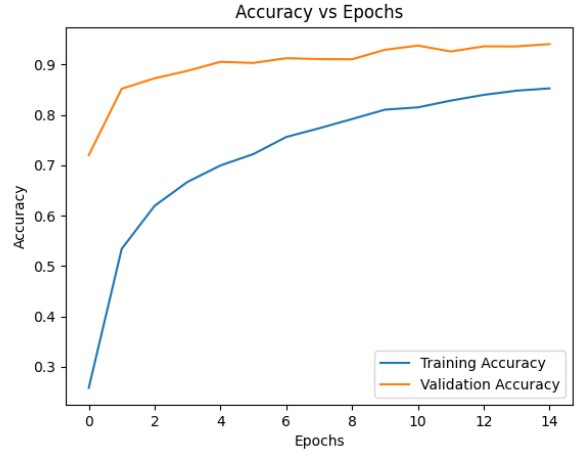


Fig 2. Dependency of accuracy on the number of iterations.

The prediction results of the first 12 images in the testing set and their corresponding labels are shown in the picture below:

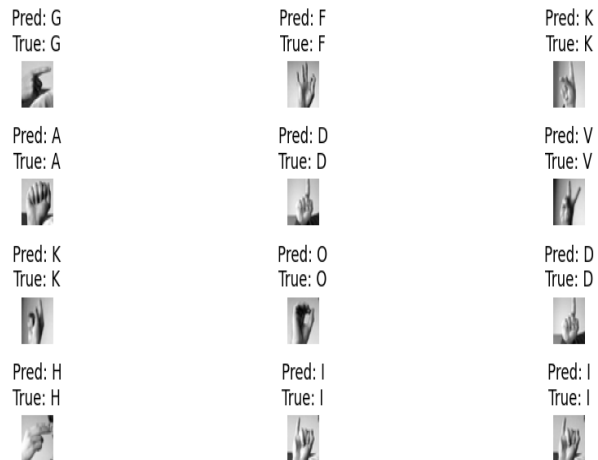


Fig 3. The predictions and labels of the first 12 testing images.

At first our model demonstrated 100% accuracy during training and near zero loss. In the testing phase, however, the loss did not decrease with iterations and accuracy didn't increase. This was a classic case of model overfitting that can be explained by the size of the dataset that we used, which is too small for a CNN model. To fix the issue, we added a dropout that randomly turned off 70% of neurons in the hidden layer on every iteration. We tried using different percentages until we found the best value. After this model improvement the overfitting disappeared and both the training and testing phases demonstrated converging loss and increasing accuracy as the number of iterations increased. As can be seen on figure 1 and figure 2, the loss function converges to around 0.3 in both training and testing phases, while the accuracy increases to around 90% in both phases. After 15 iterations the final values of loss and accuracy in the training phase were: accuracy = 0.8528, loss = 0.3647. In the testing phase the values were: accuracy = 0.9407, loss = 0.2404. Figure 3 shows the model's performance when predicting the letter on the first 12 images in the testing set. As can be seen, the classes of all 12 images were correctly predicted despite the similarities, for example, in hand signs of letters V and K.

IV. FURTHER DEVELOPMENT

Future development of our model is going to be in several directions. As mentioned in the Approach section, our model allows the addition of more filters in the convolutional layer. We plan to use fixed filters before our dynamic filters. Gaussian filter will be used to reduce noise in the images and will be followed by Sobel X and Sobel Y filters that emphasize edges. We are going to compare recognition results when adding or removing filters to see how the filters affect the overall performance of the model. The second direction involves changing hyperparameters of the model and analyzing their influence on the model's performance. So far we set our hyperparameters to the following fixed values: number of epochs = 15, learning rate = 0.001, batch size = 64, number of neurons in the hidden layer = 128, number of hidden layers = 1, number of filters in the first dynamic convolutional layer = 32, and number of filters in the second dynamic convolutional layer = 64. Finally, we are going to add one more part to our program that will take new images of hand signs and predict the letters represented by them. This part involves preparing the images for recognition by converting them to the format of the images in the Sign Language MNIST dataset that we use. The conversion would include resizing the images, making them grayscale, and possibly changing other characteristics, such as contrast, brightness, etc. One more direction for future development would be adding more classes and the corresponding datasets. The first step would be adding numbers and then adding more complex signs that represent whole words or even idioms.

When we are satisfied with the model performance results for static image recognition, we may consider extending the model to recognize continuous hand signs in videos. This might possibly involve changing the model completely, for example, switching from CNN to a Transformer model as suggested by paper [3]. Additional changes might involve using different datasets, using different techniques of splitting video feeds into frames, etc.

VIII. CONCLUSION

We have successfully created our CNN model capable of recognizing hand sign images for letters with high accuracy. Our CNN model performs Sign Language Recognition by reading an input hand sign in the form of an image and outputting it in the form of a letter. Our program implements a complete CNN model with filters, padding, and pooling in the convolutional layer. The fully connected network contains a hidden layer with 128 neurons and an output layer with 26 classes. Our recognition results have proven that the choice of the CNN model was justified. We achieved a high accuracy result of 94% using test images from the Sign Language MNIST dataset. However, the work on the project is by no means complete. There are several ways to improve our model as discussed in the Further Development section and we plan to continue working on the development of the model. We hope to achieve the same high accuracy with new hand sign images not from a dataset but taken manually.

REFERENCES

- [1] "Sign Language MNIST," [www.kaggle.com. https://www.kaggle.com/datasets/datamunge/sign-language-mnist/data](https://www.kaggle.com/datasets/datamunge/sign-language-mnist/data).
- [2] A. Abdullah, G. A. Amoudi, and H. S. Alghamdi, "Advancements in Sign Language Recognition: A Comprehensive Review and Future Prospects," *IEEE Access*, vol. 12, pp. 128871–128895, Jan. 2024, doi: <https://doi.org/10.1109/access.2024.3457692>.
- [3] U. Farooq, M. S. M. Rahim, N. Sabir, A. Hussain, and A. Abid, "Advances in machine translation for sign language: Approaches, limitations, and challenges," *Neural Comput. Appl.*, vol. 33, no. 21, pp. 14357–14399, Nov. 2021.
- [4] A. A. Barbhuiya, R. K. Karsh, and R. Jain, "CNN based feature extraction and classification for sign language," *Multimedia Tools Appl.*, vol. 80, no. 2, pp. 3051–3069, Jan. 2021.
- [5] Y. Zhang and X. Jiang, "Recent Advances on Deep Learning for Sign Language Recognition," *Computer Modeling in Engineering & Sciences*, vol. 139, no. 3, pp. 2399–2450, 2024, doi: <https://doi.org/10.32604/cmescs.2023.045731>.
- [6] H. M. Sarmad Khan, S. D. McLoughlin, and I. Murtagh, "Comparative Evaluation and Utilization of Convolutional Neural Network Architectures for Irish Sign Language Recognition," *International Journal of Combinatorial Optimization Problems and Informatics*, vol. 16, no. 1, pp. 123–131, Mar. 2025, doi: <https://doi.org/10.61467/2007.1558.2025.v16i1.550>.
- [7] A. A. Barbhuiya, R. K. Karsh, and R. Jain, "CNN based feature extraction and classification for sign language," *Multimedia Tools Appl.*, vol. 80, no. 2, pp. 3051–3069, 2021, doi: [10.1007/S11042-020-09829-Y](https://doi.org/10.1007/S11042-020-09829-Y).
- [8] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. 30th IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 2261–2269, doi: [10.1109/CVPR.2017.243](https://doi.org/10.1109/CVPR.2017.243).
- [9] H. A. Taher and S. R. M. Zeebaree, "A Critical Study of Recent Deep Learning-Based Continuous Sign Language Recognition," *The Review of Socionetwork Strategies*, vol. 19, no. 1, pp. 131–161, 2025, doi: [10.1007/s12626-025-00180-y](https://doi.org/10.1007/s12626-025-00180-y).
- [10] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986, doi: <https://doi.org/10.1038/323533a0>.